

SISTEMA DE GESTÃO

Coworking

BASES DE DADOS · LECI · 3.º ANO · APF-T

Bruno Pereira 112726 · Rafael Claro 088860

Junho 2026

O que **resolve**?

Sistema completo de gestão para um operador de coworking com múltiplos espaços físicos - do registo de cliente ao pagamento, com auditoria e regras de negócio.

- **Domínio principal**

- **Clientes**, planos (Flex / Fixo / Privado) e adesões
- Reservar **salas de reunião** à hora
- Reservar **postos** ao dia ou via adesão
- **Pagamentos** com snapshot do preço cobrado

- **Operações transversais**

- **Notificações** automáticas via triggers
- **Lista de espera** + promoção a reserva
- **Reservas recorrentes** (semanais)
- **Auditoria temporal** (history tables)

Stack: SQL Server 2019 Express · C# WinForms (.NET 8) · ADO.NET parametrizado · zero SQL injection por construção

Diagrama Entidade-Relacionamento

Cardinalidade 1 / N nas pontas (1:N, 1:1) · PK · FK · UK · supertype

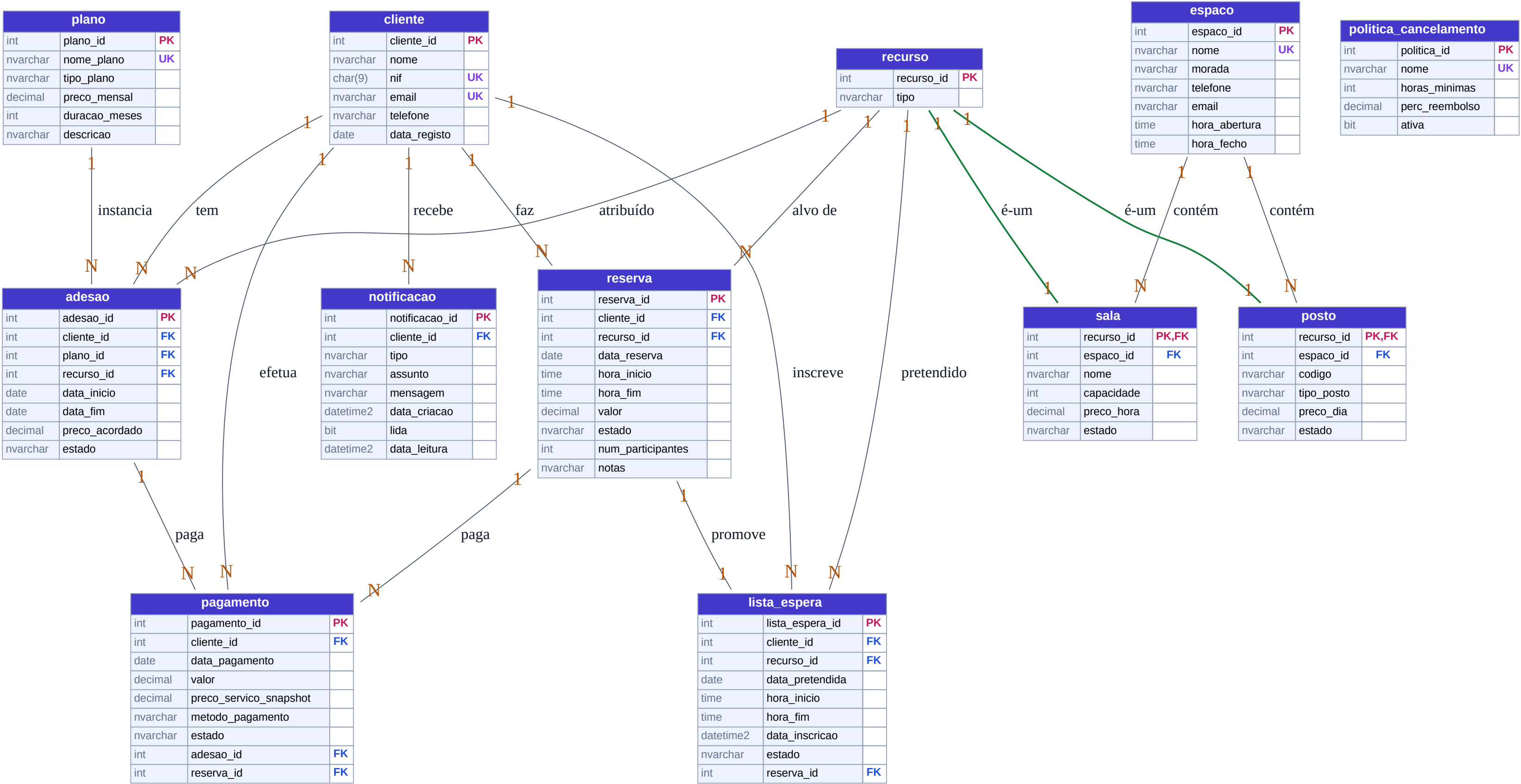
recurso

é-um

sala

/

posto



Esquema Relacional

PK sublinhado · FK ↑tabela · UNIQUE * · NULL permitido ?

```
cliente (cliente_id, nome, nif*, email*, telefone, data_registro)
plano (plano_id, nome_plano*, tipo_plano, preco_mensal, duracao_meses)
espaco (espaco_id, nome*, morada, hora_abertura, hora_fecho)
recurso (recurso_id, tipo) -- supertype
    sala (recurso_id↑recurso, espaco_id↑espaco, nome, capacidade, preco_hora)
    posto (recurso_id↑recurso, espaco_id↑espaco, codigo, tipo_posto, preco_dia)

adesao (adesao_id, cliente_id↑, plano_id↑, recurso_id↑?,
        data_inicio, data_fim, preco_acordado /* snapshot do plano */, estado)
reserva (reserva_id, cliente_id↑, recurso_id↑, data_reserva,
        hora_inicio?, hora_fim?, valor, estado, num_participantes)
pagamento (pagamento_id, cliente_id↑, adesao_id↑ XOR reserva_id↑,
            data, valor, preco_servico_snapshot, metodo, estado)
```

SQL Scripts

8 ficheiros separados por tópico em `source/SQL_Scripts/`

01 **SQL_DDL.sql**
schema · tabelas · FKs · CHECKs

03 **Triggers.sql**
16 triggers de regras de negócio

05 **Stored_procedures.sql**
SPs de negócio

07 **Indexes.sql**
índices + filtered

02 **User_defined_functions.sql**
7 UDFs (scalar + TVF)

04 **Views.sql**
11 views de consulta

06 **Temporal_tables.sql**
SYSTEM_VERSIONING

08 **SQL_DML.sql**
seed (~20 clientes · ~72 reservas)

Supertype

recurso

SQL_DDL.sql · linhas 90-115

```
CREATE TABLE recurso (  
    recurso_id INTEGER IDENTITY(1,1) PRIMARY KEY,  
    tipo NVARCHAR(10) NOT NULL  
        CHECK (tipo IN ('Sala', 'Posto'))  
);  
  
CREATE TABLE sala (  
    recurso_id INTEGER PRIMARY KEY  
        REFERENCES recurso(recurso_id) ON DELETE CASCADE,  
    espaco_id INTEGER NOT NULL REFERENCES espaco(espaco_id),  
    nome NVARCHAR(100) NOT NULL,  
    capacidade INTEGER NOT NULL CHECK (capacidade > 0),  
    preco_hora DECIMAL(10,2) NOT NULL CHECK (preco_hora > 0),  
    CONSTRAINT uq_sala_espaco_nome UNIQUE (espaco_id, nome)  
);
```

Supertype com PK = FK ao `recurso` + **CASCADE delete**. Uma única FK em `reserva.recurso_id` cobre salas e postos.

fn_recurso_disponivel

User_defined_functions.sql

```
CREATE FUNCTION fn_recurso_disponivel (  
    @recurso_id INTEGER, @data_reserva DATE,  
    @hora_inicio TIME, @hora_fim TIME  
)  
RETURNS BIT  
AS  
BEGIN  
    IF EXISTS (  
        SELECT 1 FROM reserva r  
        WHERE r.recurso_id = @recurso_id  
            AND r.data_reserva = @data_reserva  
            AND r.estado <> 'Cancelada'  
            AND ( @hora_inicio < r.hora_fim  
                AND @hora_fim > r.hora_inicio )  
    )  
        RETURN 0;      -- ocupado  
    RETURN 1;          -- disponível  
END;
```

Usada pela aplicação antes de oferecer a opção de reservar, e pelo trigger T7 antes de aceitar o INSERT.

sp_criar_reserva_sala

Stored_procedures.sql · concorrência com sp_getapplock

```
CREATE PROCEDURE sp_criar_reserva_sala
    @cliente_id INTEGER, @recurso_id INTEGER,
    @data_reserva DATE, @hora_inicio TIME, @hora_fim TIME, ...
AS BEGIN
    SET XACT_ABORT ON;
    DECLARE @lock_res NVARCHAR(255) =
        CONCAT('reserva_recurso_', @recurso_id, '_', @data_reserva);

    BEGIN TRANSACTION;

    EXEC sp_getapplock                -- evita race condition T1
        @Resource = @lock_res,
        @LockMode = 'Exclusive',
        @LockOwner = 'Transaction';

    INSERT INTO reserva (...) VALUES (...);    -- dispara T1, T2, T3, T7, T12

    COMMIT;
END;
```

sp_getapplock em modo **Exclusive · Transaction** serializa INSERTs concorrentes para o mesmo recurso/dia. Resolve a **race condition** que existiria com apenas o trigger T1.

vw_top_clientes_receita

Views.sql · ranking de clientes com RANK()

```
CREATE VIEW vw_top_clientes_receita AS
SELECT
  c.cliente_id,
  c.nome,
  COALESCE(SUM(pg.valor), 0) AS receita,
  COUNT(pg.pagamento_id) AS num_pagamentos,
  RANK() OVER (ORDER BY COALESCE(SUM(pg.valor), 0) DESC) AS posicao
FROM cliente c
LEFT JOIN pagamento pg
  ON pg.cliente_id = c.cliente_id
  AND pg.estado = 'Pago'
GROUP BY c.cliente_id, c.nome;
```

Usada no UC **Estatísticas** para ranking. `RANK()` em vez de `ROW_NUMBER()` permite **empates correctos**. `LEFT JOIN` garante que clientes sem pagamentos aparecem com receita = 0.

Snapshot do preço cobrado

Resposta directa à pergunta "qual o preço?" no pagamento

Triggers.sql · trg_pagamento_valor_correto

```
CREATE TRIGGER trg_pagamento_valor_correto
ON pagamento AFTER INSERT, UPDATE AS
BEGIN
    IF EXISTS (
        SELECT 1 FROM inserted i
        JOIN reserva r ON i.reserva_id = r.reserva_id
        WHERE i.preco_servico_snapshot <> r.valor )
    BEGIN
        ROLLBACK;
        THROW 50005, 'Snapshot não corresponde ao valor da reserva.', 1;
    END
    -- (idem para adesão)
END;
```

preco_servico_snapshot + **CHECK constraint** + este trigger garantem que o valor cobrado fica **imutável** mesmo que o preço do serviço mude depois - **3 níveis de defesa**.

Filtered indexes

Indexes.sql · otimizações chave

```
-- Reserva: detecção de sobreposições (T1) por recurso/dia
CREATE INDEX idx_reserva_recurso
  ON reserva (recurso_id, data_reserva, hora_inicio, hora_fim);

-- Pagamentos: filtered indexes evitam linhas NULL (~50% storage)
CREATE INDEX idx_pagamento_adesao
  ON pagamento (adesao_id)
  WHERE adesao_id IS NOT NULL;

CREATE INDEX idx_pagamento_reserva
  ON pagamento (reserva_id)
  WHERE reserva_id IS NOT NULL;
```

`SET STATISTICS IO ON` confirma: T1 sobreposição passa de **Table Scan** → **Index Seek**. Filtered indexes em pagamento reduzem ~50% storage porque metade das linhas tem o ID a NULL.

Temporal Tables

Temporal_tables.sql · SYSTEM_VERSIONING

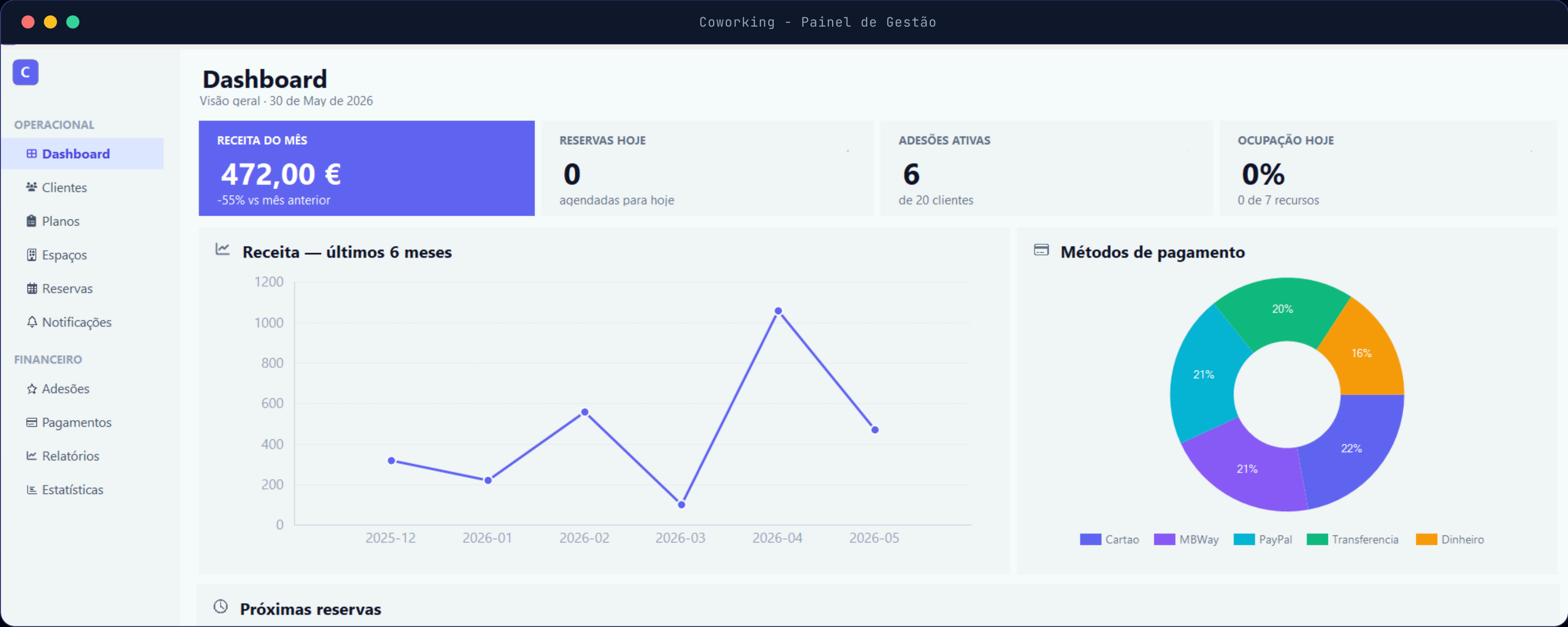
```
ALTER TABLE adesao    SET (SYSTEM_VERSIONING = ON);
ALTER TABLE reserva   SET (SYSTEM_VERSIONING = ON);
ALTER TABLE pagamento SET (SYSTEM_VERSIONING = ON);
```

-- Reconstituir estado num instante passado:

```
SELECT *
FROM   reserva
FOR    SYSTEM_TIME AS OF '2026-05-01 00:00:00';
```

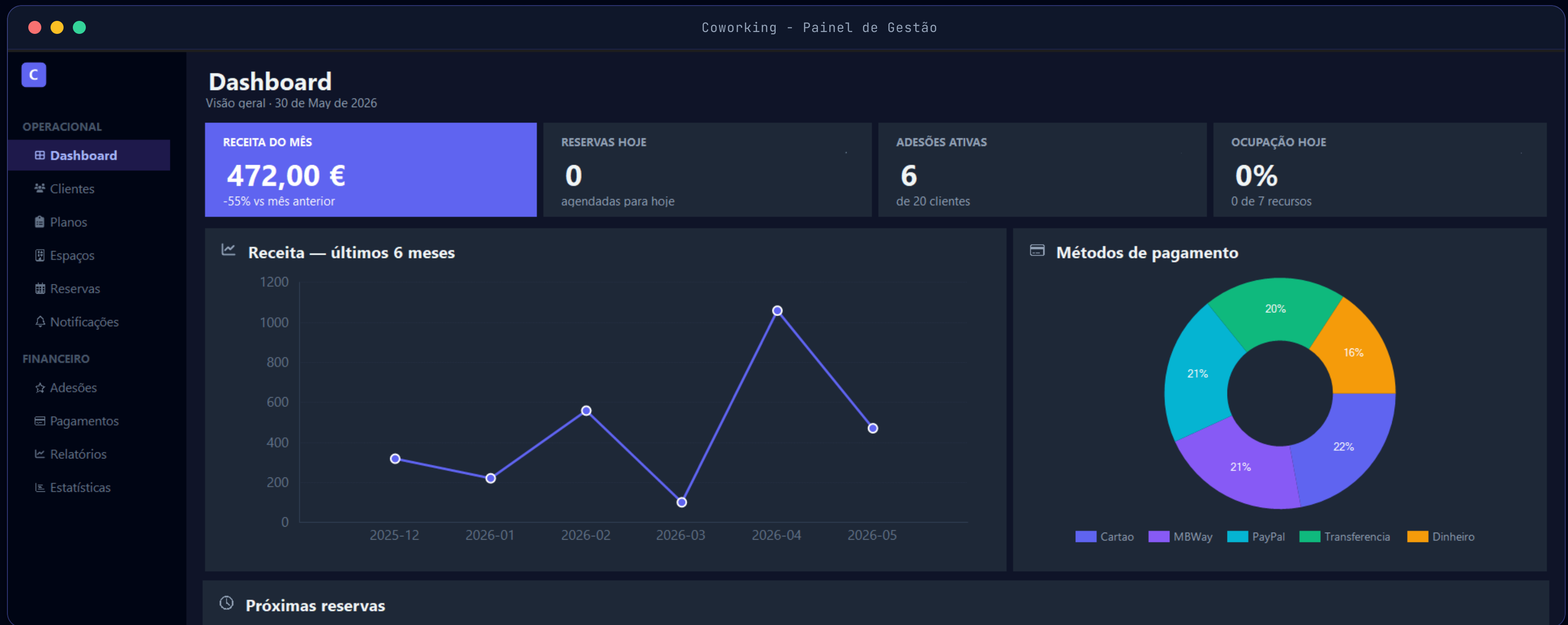
Cada UPDATE/DELETE escreve a versão anterior em `<tabela>_history` automaticamente. **Sem trigger adicional**, sem schema manual de histórico. Suporta **FOR SYSTEM_TIME ALL / AS OF / BETWEEN**.

Dashboard light



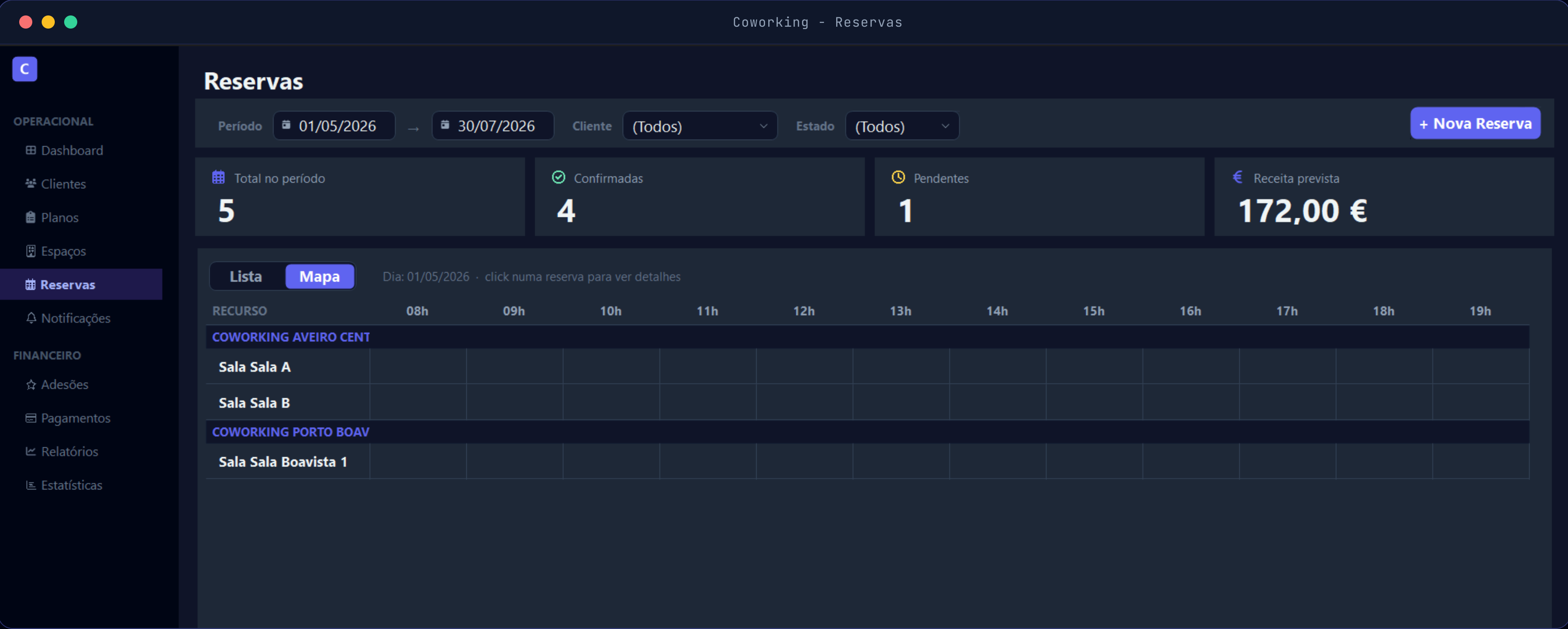
KPIs operacionais · gráficos Receita Mensal e Métodos de Pagamento · próximas reservas

Dashboard dark

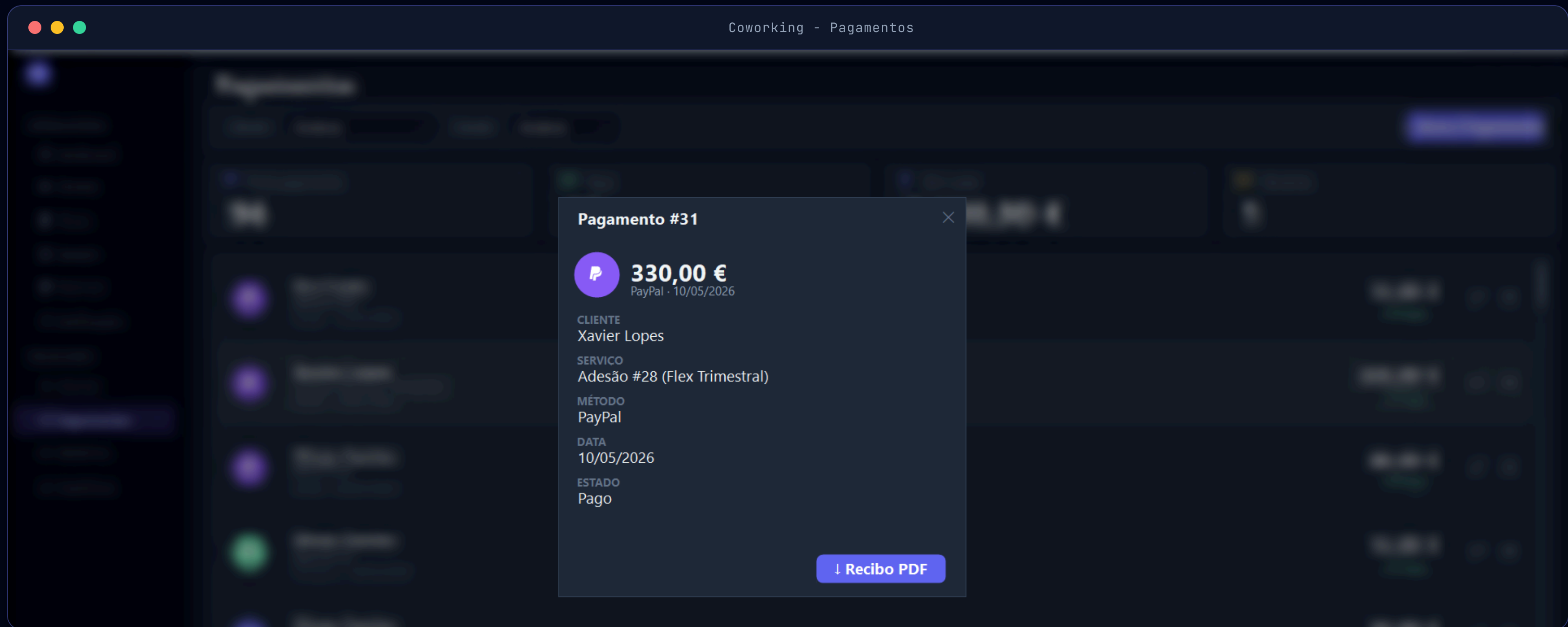


Toggle light ↔ dark **em tempo real** · preferência persiste em %LocalAppData%

Reservas · Tab Mapa



Pagamentos & Recibo PDF



Detalhe read-only com botão "↓ Recibo PDF" · gerado via PdfSharp 6.1.1 com snapshot do preço

Estatísticas



DEMO

Demonstração ao vivo da aplicação

Em números

12

Tabelas em BCNF

16

Triggers de negócio

18+

Stored Procedures

11

Views de consulta

7

UDFs

10

UserControls C#

Regras de negócio em **3 níveis** · concorrência segura com **sp_getapplock** · auditoria via **temporal tables** · correcções do prof. **resolvidas** (#1 snapshot · #2 reserva associativa)

Obrigado